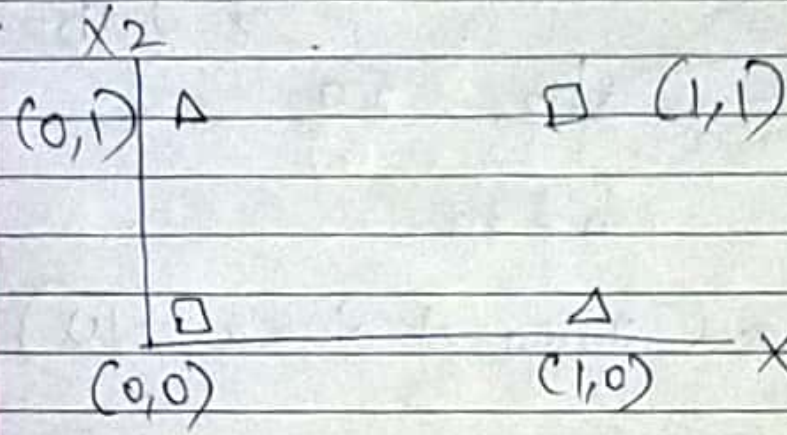
XOR problem

X_1	X_2	Ans y
0	0	0
0	1	1
1	0	1
1	1	0

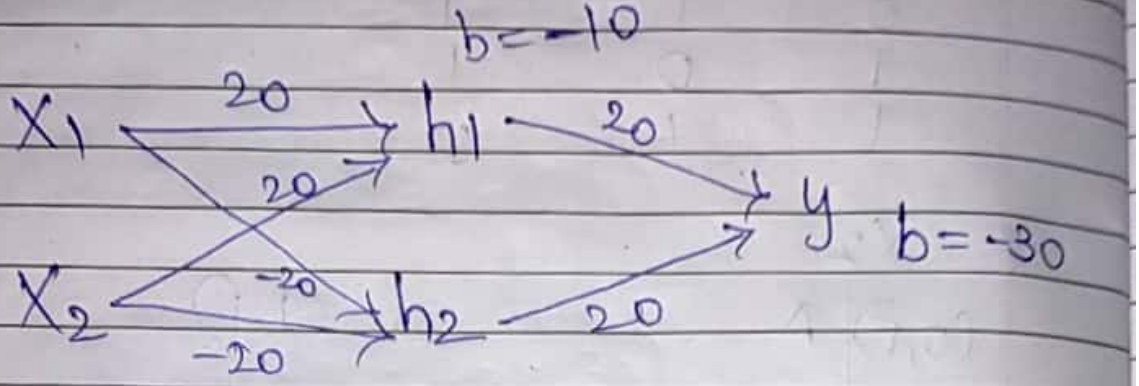


These types of data of XOR we need neural network as ML will not be able to solve as problem requires two hyperplanes.

For Non-linear data Deep Learning is required.

AND problems can be easily solved using ML, whereas XOR problems are solved using DL & ML fails in XOR problems.

*. Solving XOR problem with a Neural Network.



$h_1 = \sigma(20x_1 + 20x_2 - 10)$

$h_2 = \sigma(-20x_1 - 20x_2 + 30)$

$y = \sigma(20h_1 + 20h_2 - 30)$

Inputs :

$(0, 0)$
$x_1\ x_2$

$h_1 = \sigma(-10)$

$h_2 = \sigma(30)$

$\sigma = h_1 = \frac{1}{1+e^{-h_1}} = \frac{1}{1+e^{10}}$

$= \frac{1}{1+22,026.466}$

$= \frac{1}{22027.466}$

$= 0.000045 = 0$

$$h_2 = \frac{1}{1+e^{-30}}$$

$$= \frac{1}{1} = 1$$

$$y = \sigma(20(0.000045) + 20(1) - 30)$$

$$= \sigma(0.0009 + 20 - 30)$$

$$= \sigma(-9.9991)$$

$$= \frac{1}{1+e^{9.9991}}$$

$$y = \sigma(20(0) + 20(1) - 30)$$

$$= \sigma(-10)$$

$$= 0$$

Input 2

(0, 1)

$$h_1 = \sigma(20 - 10)$$

$$= \sigma(10)$$

$$= 1$$

$$h_2 = \sigma(-20 + 30)$$

$$= \sigma(10)$$

$$= 1$$

$$y = \sigma(20 + 20 - 30)$$

$$= \sigma(10)$$

$$= 1$$

Input 3

(1, 0)

$h_1 = \sigma(20 - 10)$

$= \sigma(10)$

$= 1$

$h_2 = \sigma(-20 + 30)$

$= \sigma(10)$

$= 1$

$y = (20 + 20 - 30)$

$= \sigma(10)$

$= 1$

Input 4

(1, 1)

$h_1 = \sigma(20 + 20 - 10)$

$= \sigma(30)$

$= 1$

$h_2 = \sigma(-20 - 20 + 30)$

$= \sigma(-40 + 30)$

$= \sigma(-10)$

$= 0$

$y = \sigma(20 - 30)$

$= \sigma(-10)$

$= 0$

→ Invariance

It can be achieved using Pooling.

Input can be at any position.

Output will be same

Input changes output

(Initial Output as input will be given)

→ Equivariance

$$h(I) = I(h)$$

Filter on image = Image on filter.

→ Translation will work.

Affine transformation, scaling & rotation will not work.

(Filtered output will be given)

Convolutional

Sum

(I) Input = 6×6

(f) filter = 3×3

(OI) = 4×4

Output Image

⇒ Input = n×n

Filtersize = f×f

Output = (n-f+1) × (n-f+1)

= (6-3+1) × (6-3+1)

= 4×4

Padding - 1) Valid
2) Same

Disadvantages :

1. Every time we apply a convolutional operation, the size of image shrinks.
2. Pixels present at the corner of image are used only a few number of times during convolutional.

Hence we don't focus too much on the corners since that can lead to information loss.

⇒ Padding are of two types:

1.) Valid

2.) Same

Valid - It means no padding (Output - shrink)

Same - Here we apply padding so that the output size is the same as the input size.

Same padding = P

input = n x n

F = f x f

$O = (n + 2P - f + 1) \times (n + 2p - f + 1)$

$(n + 2p - f + 1) = n$

$2p - f + 1$

$p = \frac{f - 1}{2}$

2P = Because padding is done both side top-bottom left-right

→ Overcome disadvantages

Padding is used.

* Stride

Stride = 8
Padding = p
I = n x n f = f x f

$$\text{Output} = \left[\frac{(n+2p-f)}{s} + 1\right] \times \left[\frac{n+2p-f}{s} + 1\right]$$

I 6 x 6

$$\frac{n+2p-f}{s} + 1$$

s = 1

$$= \frac{6+2(0)-3}{1} + 1$$

$$= \frac{3}{1} + 1 = 4$$

s = 2

$$\frac{6+0-3}{2} + 1$$

$$= \frac{3}{2} + 1 = 1.5 + 1$$

$$= 2.5 \approx 2$$

Advantage of Stride

- Stride helps to reduce size of image
- Helps to avoid overlapping of pixels. Thus focusing on particular useful features.

* Convolution over Volume

Input $6 \times 6 \times 3$
Filter $3 \times 3 \times 3$

(Height x Width x channel)

Channel
1- Grayscale
3- Color

1) No. of parameters depends on filter size
2) Input channel & filter channel should be same. (Thumb Rule)

$6 \times 6 \circledast 3 \times 3 = 4 \times 4$

$6 \times 6 \times 3 \circledast (3 \times 3 \times 3) = 4 \times 4 \times 2$

Channel will changed acc. to filters applied.

4x4 } 4x4x2
4x4

Input = $n \times n \times n_c$ ← channel

filter = $f \times f \times n_c$

Output = $\left[\frac{(n + 2p - f)}{s} + 1\right] \times$

$\left[\frac{(n + 2p - f)}{s} + 1\right] \times n_c'$

where
n_c' is the no. of channels in input & filter
while n_c' is the no. of filters.

→ Pooling Layers

Output of Con layer is passed to pooling layer

Con - I - 6 X 6
Fil - 3 X 3
O - 4 X 4 ← Feature Map

1.) Max Pooling

2.) Average Pooling

Output of Con layer

F.M.

2	10	15	10
15	25	20	5
10	5	12	2
50	18	3	4

4×4

16 features

Max pooling S = 2
filter = 2×2

In Average
2+10+15+25 / 4

25	20
50	12

2×2

4 features

Pooling layer

- Reduces no. of parameters (learnable)
- Important features are extracted
- Reduce size of image

Down-size the image with stride 2

General Formula of Pooling

$$\left[\frac{nh - f}{s} + 1\right] \times \left[\frac{nw - f}{s} + 1\right] \times nc$$

$nh - nw$ = Input Image's height & weidth

f = filter

s = stride

nc = channel

Sum:

Input - $6 \times 6 \times 3$

filter - $3 \times 3 \times 3$

output - 4×4

$z^{[1]} = w^{[1]} * a^{[0]} + b^{[1]}$ (filter)

w - weight

a - input

$A = \sigma(z^{[1]})$

# of Parameters

Calculation of no. of Parameters

No. of parameters for each filter
$= 3 \times 3 \times 3 = 27$

There will a biased term for each filter

Thus, there is total parameters for filter
$= 27 + 1 = 28$

Let, applying five filters,
the total parameters for that layer

$28 \times 5 = 140$

⇒ CNN Architecture for digit recognition

filter $= 28 \times 28 \times 8$
Apply 8 filters / channels

Filters change after conv. layer

Chanel / No. of filter won't change after pooling.

In conv2, we will assume no. of filters = 16

Fully connected layers

No. of neurons, filters in convolutional are decided / assumed by us.

Input & output layers are fix

No. of class are equal to output.

0 to 9 digits so 10 is output.

No. of parameters ($Conv_1$)

$= [(5,5,3) + 1] * 8$

$= 76 * 8$

$= 608$

No. of parameters ($Conv_2$)

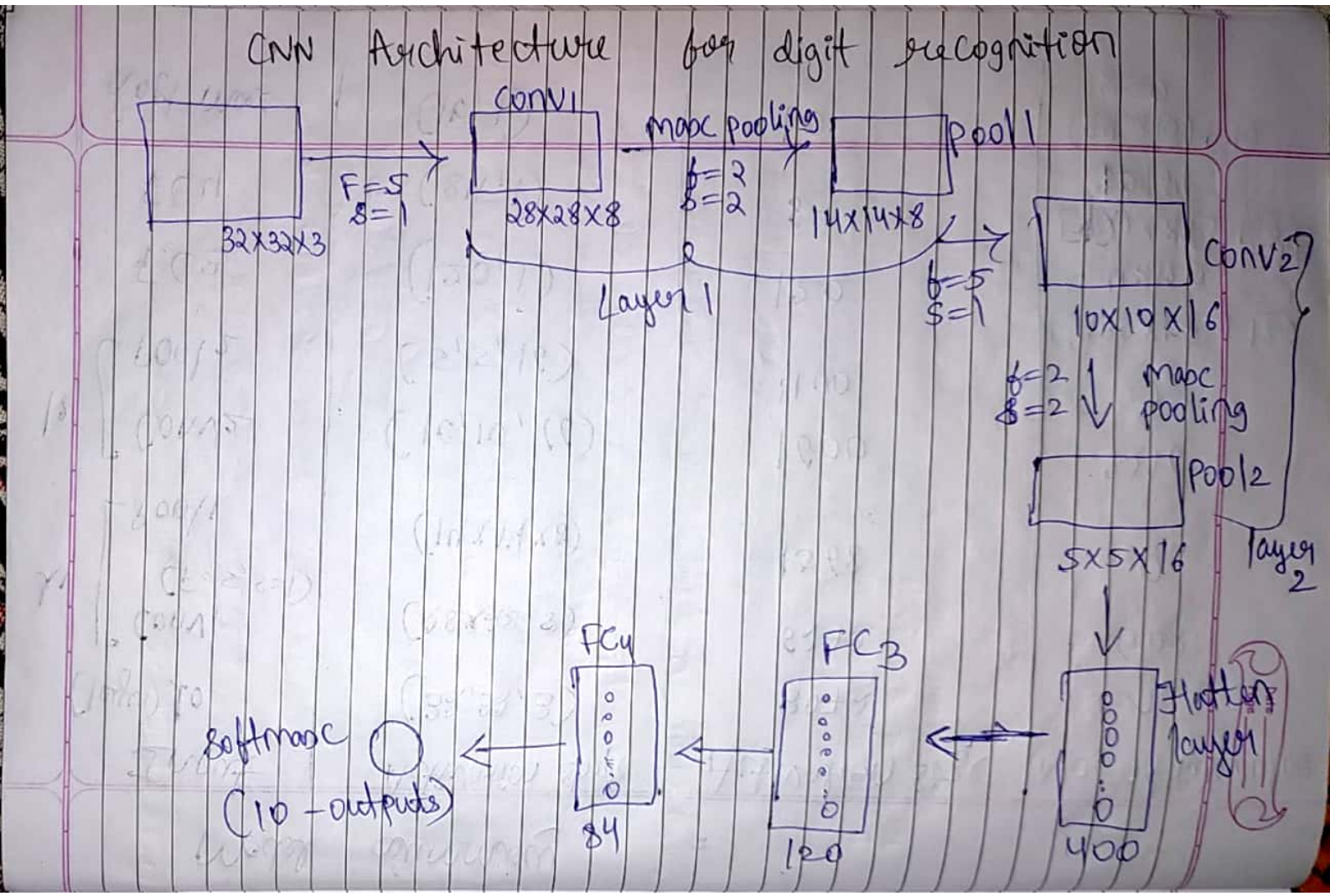
$= [(5,5,8) + 1] * 16$

$= 200 + 1 * 16$

$= 3216$

Model Summary

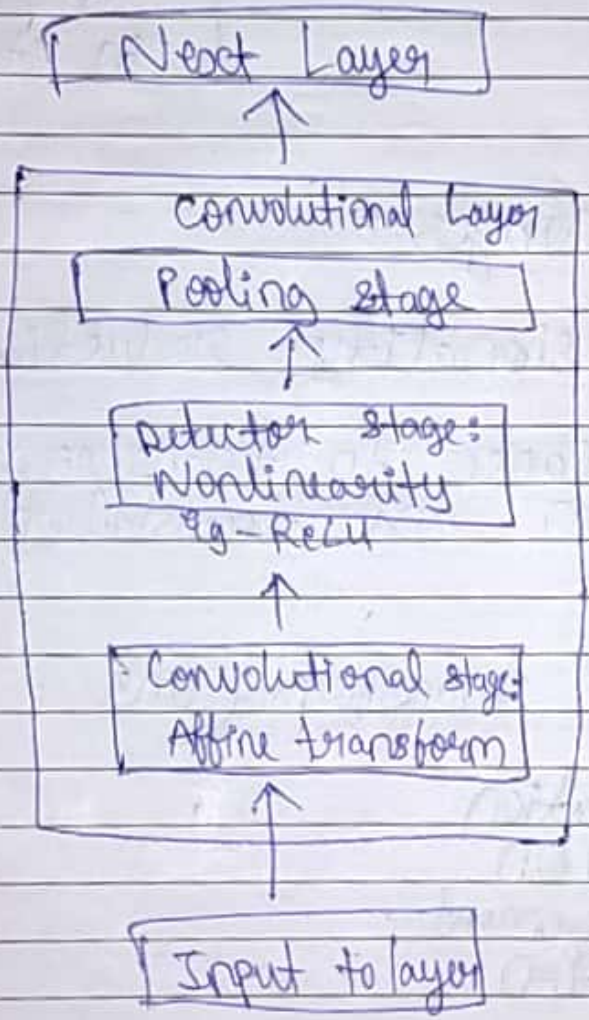
Layer (type)	Activation Shape	Activation Size	No. of Parameters
Input (l_0)	(32, 32, 3)	3072	0
l_1 [Conv$_1$ (F=5, S=1)	(28, 28, 8)	6272	608
l_1 [Pool$_1$	(14 x 14 x 8)	1568	0
l_2 [Conv$_2$	(10, 10, 16)	1600	3216
l_2 [Pool$_2$	(5, 5, 16)	400	0
FC3	(120, 1)	120	[400 x 120] + 120 = 48120
FC4	(84, 1)	84	[120 x 84] + 84 = 10164
Soft max	(10, 1)	10	(84 x 10) + 10 = 850



Pooling in Convolutional Networks

It replaces output at a location with a summary statistic of nearby outputs

It is a subsampling step.



Stage 1 : Convolutional
Stage 2 : Detector
Stage 3 : Pooling

Two terminologies for typical CNN layer

1. Net is viewed as a small no. of complex layers, each layer having many stages.	2. Net is viewed as a larger no of simple layers. Every processing step is a layer in its own right.

Advantages :

1. Dimensionality reduction (Downsampling)
2. Invariance to transformations of rotation and translation

Linear transformations

1. Translation
2. Rotation
3. Enlargement
4. Reflection

Pooling Functions types :

1. max
2. min
3. Average
4. Sum
5. Weighted average